universität ulm

Numerical Optimization
Winter semester 2025/26

Exercise sheet 2 for Numerical Optimization

Manuel Caipo
Degree program: M.Sc. Computational Science Engineering
Enrollment number: 30783212

Instructors: Dr. Michael Lehn, Tobias Born

28. October 2025 - 89081 Ulm

Chapter 1

Solutions Exercise sheet 2

1.1 Exercise 1

1.1.1 Part a)

The Gauss–Newton (GN) method uses the Hessian approximation

$$H \approx J^\top J, \tag{1.1}$$

which neglects the second–order derivative terms of the residual functions. The Levenberg–Marquardt (LM) method modifies this approximation by adding a damping term:

$$H_{\mathrm{LM}} \approx J^\top J + \mu^2 I. \tag{1.2}$$

For $\mu = 0$, we recover the Gauss–Newton method; for large μ, the step becomes similar to a gradient–descent step, making the method more conservative.
Regarding the dimensions, the step $s_k \in \mathbb{R}^n$ has the same dimension as x_k. The residual vector is $F(x_k) \in \mathbb{R}^m$ and the Jacobian matrix is $J(x_k) \in \mathbb{R}^{m\times n}$. Higher–order tensors are not required for the basic algorithm.

The Levenberg–Marquardt step is defined as

$$s_k = -\left(J^\top J + \mu^2 I\right)^{-1} J^\top F. \tag{1.3}$$

The corresponding normal equations are

$$\left(J^\top J + \mu^2 I\right) s = -J^\top F. \tag{1.4}$$

This system is exactly equivalent to the following linear least squares problem in the variable s:

$$\boxed{\min_{s\in\mathbb{R}^n} \|Js + F\|_2^2 + \mu^2\|s\|_2^2.} \tag{1.5}$$

Hence, the Levenberg–Marquardt update can be interpreted as solving a modified linear least squares problem with an additional damping term that stabilizes the solution.

1.1.2 Part b)

We want to show that the Levenberg–Marquardt equation

$$(J^\top J + \mu^2 I)\, s = -J^\top F \tag{1.6}$$

is the normal equation of the penalized least squares problem

$$\min_s \|Js + F\|_2^2 + \mu^2\|s\|_2^2, \tag{1.7}$$

where $J = F'(x_k)$, $F = F(x_k)$, and $\mu \geq 0$.
Expand the first term $\|Js + F\|^2$.
We first expand it explicitly:

$$\|Js + F\|^2 = (Js + F)^\top (Js + F) = (Js)^\top (Js) + (Js)^\top F + F^\top (Js) + F^\top F. \tag{1.8}$$

Now, we simplify each part:

$$(Js)^\top (Js) = s^\top J^\top Js, \quad (Js)^\top F = s^\top J^\top F, \quad F^\top (Js) = s^\top J^\top F, \quad F^\top F \text{ is constant.}$$

Therefore,

$$\|Js + F\|^2 = s^\top J^\top Js + 2\, s^\top J^\top F + F^\top F. \tag{1.9}$$

Add the penalty term.
The regularization term is

$$\mu^2 \|s\|^2 = \mu^2 s^\top s = s^\top (\mu^2 I) s. \tag{1.10}$$

Hence, the full objective function is

$$\Phi(s) = s^\top (J^\top J + \mu^2 I) s + 2\, s^\top J^\top F + F^\top F. \tag{1.11}$$

Taking the gradient with respect to s and setting it to zero gives

$$\nabla \Phi(s) = 2(J^\top J + \mu^2 I) s + 2 J^\top F = 0. \tag{1.12}$$

Thus,

$$(J^\top J + \mu^2 I)\, s = -J^\top F. \tag{1.13}$$

This shows that the Levenberg–Marquardt step is the solution of the modified least squares problem

$$\min_s \|Js + F\|^2 + \mu^2 \|s\|^2, \tag{1.14}$$

which corresponds to the Gauss–Newton problem from sheet 1 plus an additional quadratic penalty term on s. The parameter μ controls the transition between the Gauss–Newton method ($\mu = 0$) and a more conservative, gradient-like step for large μ.

Part c)

Damping property for the Levenberg–Marquardt method (case $+\mu^2 I$) We want to prove the **damping property** of the Levenberg–Marquardt (LM) method, that is,

$$\|s\|_2 \leq \frac{\|J^\top F\|_2}{\mu^2}. \tag{1.15}$$

This inequality shows that the step length $\|s\|$ becomes smaller as μ increases, which explains why μ acts as a damping parameter: it slows down the update and keeps the iteration stable.

1. Starting point In the LM method, the update direction s is defined by the linear system

$$(J^\top J + \mu^2 I)\, s = -J^\top F. \tag{1.16}$$

For the derivation, the sign on the right-hand side is not relevant, since the norm of s does not depend on it. Let us define

$$y := J^\top F, \tag{1.17}$$

so that the equation can be written as

$$(J^\top J + \mu^2 I)\, s = y. \tag{1.18}$$

2. Multiply by $s^\top$ We now multiply both sides by $s^\top$. This is a standard trick in linear algebra to obtain an "energy" expression:

$$s^\top (J^\top J + \mu^2 I)\, s = s^\top y. \tag{1.19}$$

Expanding the left-hand side gives

$$s^\top J^\top J s + \mu^2 s^\top s = s^\top y. \tag{1.20}$$

3. Observe the positive term The first term is always non-negative, because

$$s^\top J^\top J s = (Js)^\top (Js) = \|Js\|^2 \geq 0. \tag{1.21}$$

Hence we can write

$$\underbrace{\|Js\|^2}_{\geq 0} + \mu^2 \|s\|^2 = s^\top y \quad \Rightarrow \quad \mu^2 \|s\|^2 \leq s^\top y. \tag{1.22}$$

4. Apply the Cauchy–Schwarz inequality Recall the basic inequality for any vectors a, b:

$$|a^\top b| \leq \|a\|\, \|b\|. \tag{1.23}$$

Applying it here with $a = s$ and $b = y$ gives

$$|s^\top y| \leq \|s\|\, \|y\|. \tag{1.24}$$

Combining this with the previous inequality yields

$$\mu^2 \|s\|^2 \leq |s^\top y| \leq \|s\|\, \|y\|. \tag{1.25}$$

5. Divide by $\|s\|$ If $s = 0$, the inequality is trivially satisfied. Otherwise, dividing both sides by $\|s\|$ gives

$$\mu^2 \|s\| \leq \|y\|. \tag{1.26}$$

Recalling that $y = J^\top F$, we obtain

$$\boxed{\|s\| \leq \frac{\|J^\top F\|}{\mu^2}}. \tag{1.27}$$

6. Interpretation This inequality formally proves the *damping property* of the LM method. As μ increases, the matrix $(J^\top J + \mu^2 I)$ becomes larger and more dominant, so its inverse $(J^\top J + \mu^2 I)^{-1}$ becomes smaller. Consequently, the step s decreases in magnitude:

- Large $\mu \Rightarrow$ small step $\Rightarrow$ stable but slow convergence.
- Small $\mu \Rightarrow$ larger step $\Rightarrow$ faster but possibly unstable convergence.

In intuitive terms, μ^2 acts as a quadratic brake: the larger μ, the harder it is for the algorithm to take big steps, and the more "controlled" and conservative the iteration becomes.

1.1.3 Part d)

Gauss–Newton and Levenberg–Marquardt: Invertibility Condition
In the Gauss–Newton method, the update step is defined as

$$s_{\mathrm{GN}} = -\left(J^\top J\right)^{-1} J^\top F. \tag{1.28}$$

To compute this step, the matrix $J^\top J$ must be invertible. This is only possible if the Jacobian $J \in \mathbb{R}^{m\times n}$ has full column rank, i.e.

$$\operatorname{rank}(J) = n. \tag{1.29}$$

If J does not have full rank (e.g., its columns are linearly dependent), then $(J^\top J)^{-1}$ does not exist, and the Gauss–Newton step cannot be computed.

In contrast, the Levenberg–Marquardt (LM) method modifies the system as follows:

$$s_{\mathrm{LM}} = -\left(J^\top J + \mu^2 I\right)^{-1} J^\top F, \qquad \mu > 0. \tag{1.30}$$

The additional term $\mu^2 I$ acts as a *regularization* that stabilizes the matrix.

Mathematically, $J^\top J$ is symmetric and positive semi-definite, meaning it has eigenvalues

$$\lambda_i \geq 0, \quad i = 1, \ldots, n. \tag{1.31}$$

A matrix is invertible if and only if none of its eigenvalues are zero:

$$A \text{ is invertible} \iff \lambda_i \neq 0 \text{ for all } i. \tag{1.32}$$

If any $\lambda_i = 0$, then $J^\top J$ is singular (non-invertible). However, by adding the damping term $\mu^2 I$, the eigenvalues of the modified matrix become

$$\lambda_i' = \lambda_i + \mu^2, \tag{1.33}$$

which are strictly positive since $\mu^2 > 0$. Therefore,

$$J^\top J + \mu^2 I \tag{1.34}$$

is always invertible, even when J does not have full rank.

The Gauss–Newton method requires $\operatorname{rank}(J) = n$ to ensure invertibility, while the Levenberg–Marquardt method guarantees invertibility for any $\mu > 0$, making it numerically more stable and robust against rank deficiencies.

1.1.4 Part e)

```
function [X_hist, mu_hist] = levenberg_marquardt(x0, F, dF, mu0, beta0, beta1, tol,
    maxit)
    xk = x0(:);
    n  = numel(xk);
    X_hist  = zeros(n, maxit+1);
    X_hist(:,1) = xk;
    mu_hist = zeros(maxit,1);
    mu = max(mu0,0);
    k  = 0;
    Fk = F(xk);         % m x 1
    Jk = dF(xk);        % m x n
    JT = Jk.';          % n x m
    g  = JT*Fk;         % n x 1
    H  = JT*Jk;         % n x n
    for outer = 1:maxit
        I = eye(n);
        max_inner = 20;
        eps_mu = -Inf;
        for t = 1:max_inner
            % LM step: (J'J + mu^2 I) s = -J'F
            s = -(H + (mu^2)*I) \ g;

            % gain ratio indicator epsilon_mu
            F_lin = Fk + Jk*s;
            Fxks  = F(xk + s);
            num = norm(Fk)^2 - norm(Fxks)^2;
            den = norm(Fk)^2 - norm(F_lin)^2;
            if den <= 0
                eps_mu = -Inf;              % force mu increase
            else
                eps_mu = num / den;
            end
            if eps_mu > beta0
                break;                      % acceptable step
            else
                mu = 2*mu;                  % reject and increase damping
            end
        end

        if eps_mu >= beta1
            mu = mu/2;
        end
        xk_new = xk + s;
        k = k + 1;
        X_hist(:,k+1) = xk_new;
        mu_hist(k)    = mu;
        % stop criterion by step size
        if norm(s,2) < tol
            break;
        end
        xk = xk_new;
        Fk = F(xk);
        Jk = dF(xk);
        JT = Jk.';
        g  = JT*Fk;
        H  = JT*Jk;
    end

    X_hist  = X_hist(:,1:k+1);
    mu_hist = mu_hist(1:k);
end
```

1.1.5 Part f)

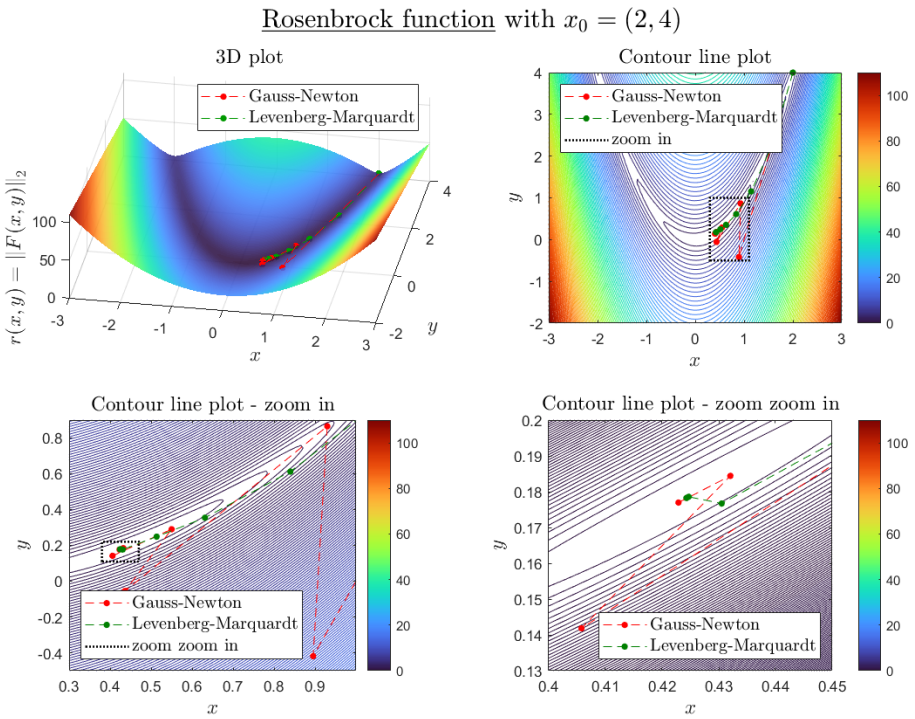

Figure 1.1: Comparison of Gauss-Newton vs. Levenberg-Marquardt methods minimizing the Rosenbrock function, starting at $x_0 = (2, 4)$.

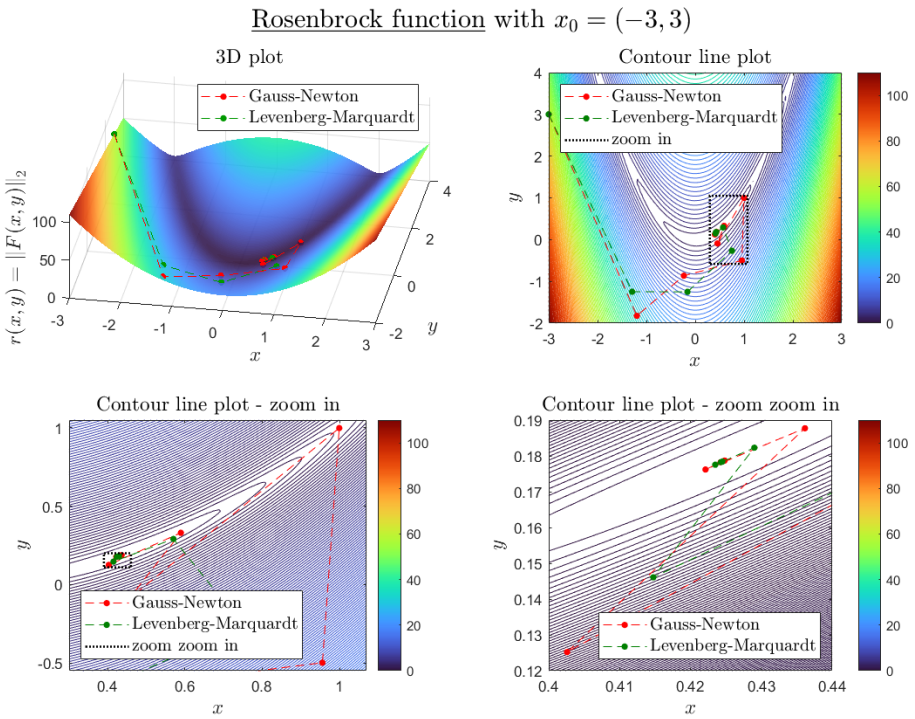

Figure 1.2: Comparison of Gauss-Newton vs. Levenberg-Marquardt methods minimizing the Rosenbrock function, starting at $x_0 = (-3, 3)$.

1.2 Exercise 2

1.2.1 Part a)

```
function [Xks] = newton_raphson(x0, F, dF, ddF, tol, maxit)

    Xks = zeros(1, maxit + 1);
    Xks(1) = x0;
    for k = 1:maxit

        xk   = Xks(k);
        Fk   = F(xk);
        dFk  = dF(xk);
        ddFk = ddF(xk);

        % NewtonRaphson
        sk = - (dFk * dFk + Fk * ddFk) \ (dFk * Fk);

        if abs(sk) < tol
            Xks(k + 1) = xk + sk;
            Xks(k + 2:end) = [];  % Trim unused preallocated space
            return
        end

        Xks(k + 1) = xk + sk;
    end

end
```

1.2.2 Part b)

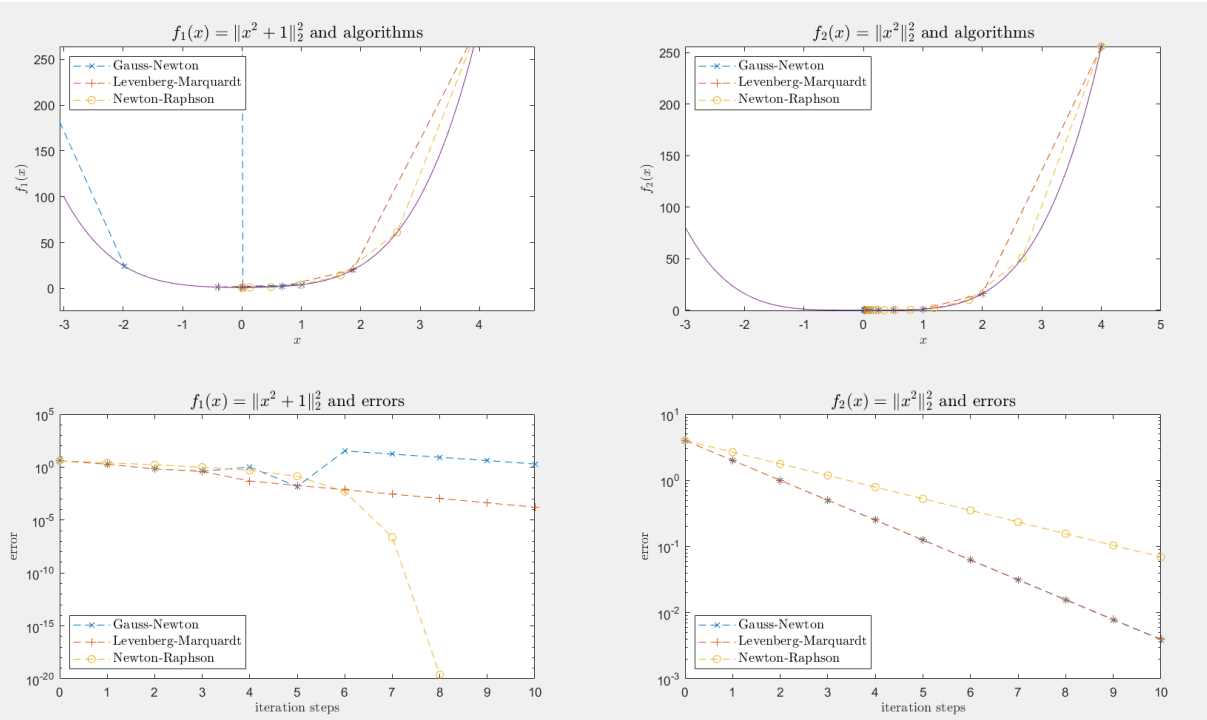

Figure 1.3: Comparison of Newton-raphson Gauss-Newton vs. Levenberg-Marquardt methods

1.3 Exercise 3

1.3.1 Part a)

```
1
2
3  function[g_best] = particle_swarm_optimization(f, p0, v0, inertia, c_p, c_g, max_it,
       max_noupd, p_plot, v_plot, g_plot, t_plot, vmax)
4
5  % set initial values
6  p_best = p0;
7  g_best = [];
8  f_best = Inf;
9  p = p0;
10 v = v0;
11 k = 0;
12 is_upd = false;
13 num_noupd = 0;
14 num_part = size(p0,2);
15
16 % perform at most max_it iteration steps
17 while k < max_it
18
19     % update g_best
20     is_upd = false;
21     f_pbest = arrayfun(@(j) f(p_best(1,j), p_best(2,j)), 1:num_part);
22     [f_cur_best, idx_best] = min(f_pbest);
23     if f_cur_best < f_best
24         f_best = f_cur_best;
25         g_best = p_best(:, idx_best);
26         is_upd = true;
27     end
28
29     % if new global best
30     if is_upd
31         num_noupd = 0;
32     else % if no new global best
33         num_noupd = num_noupd + 1;
34         % terminate if too many iteration steps without new global best
35         if num_noupd >= max_noupd
36             plot_it(p, v, g_best, k, f_best, p_plot, v_plot, g_plot, t_plot, vmax);
37             break;
38         end
39     end
40
41     % new velocities
42     r1 = rand(size(v));
43     r2 = rand(size(v));
44     v = inertia.*v + c_p.*r1.*(p_best - p) + c_g.*r2.*(g_best - p);
45
46     % plot positions and velocities
47     plot_it(p, v, g_best, k, f_best, p_plot, v_plot, g_plot, t_plot, vmax);
48
49     % update positions
50     p = p + v;
51
52     % update all particles' personal best
53     f_p = arrayfun(@(j) f(p(1,j), p(2,j)), 1:num_part);
54     improve = f_p < f_pbest;
55     if any(improve)
56         p_best(:, improve) = p(:, improve);
57         f_pbest(improve) = f_p(improve);
58     end
```

```
59
60      k = k+1;
61
62  end
63
64  end
65
66
67  % only for plotting, not relevant
68  function [] = plot_it(p, v, g_best, k, f_best, p_plot, v_plot, g_plot, t_plot, vmax)
69      set(p_plot, 'XData', p(1,:), 'YData', p(2,:));
70      set(v_plot, 'XData', p(1,:), 'YData', p(2,:), 'UData', v(1,:)/vmax, 'VData', v
        (2,:)/vmax);
71      set(g_plot, 'XData', g_best(1), 'YData', g_best(2));
72      set(t_plot, 'String', ['Iteration step: ', num2str(k), ', global best: ',
        num2str(f_best)]);
73      pause(0.01);
74  end
```

1.3.2 Part b)

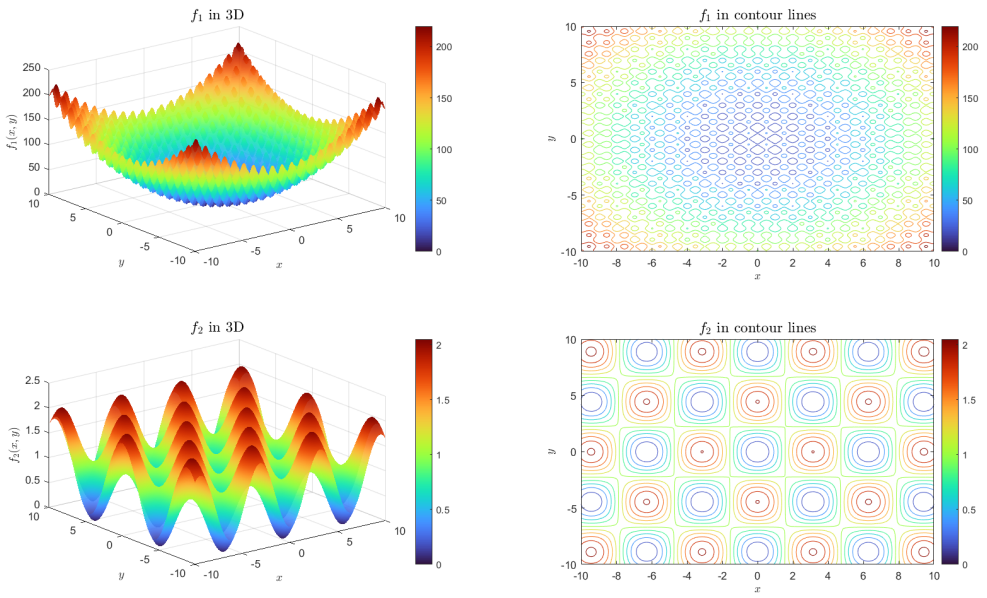

Figure 1.4: Functions f1,f2

1.3.3 Part c)

The PSO algorithm does not always return exactly the same global minimum at $(0, 0)$ due to its stochastic nature. However, in every run it converges very close to this point, showing small variations around the true minimum value $f_1(0, 0) = 0$.

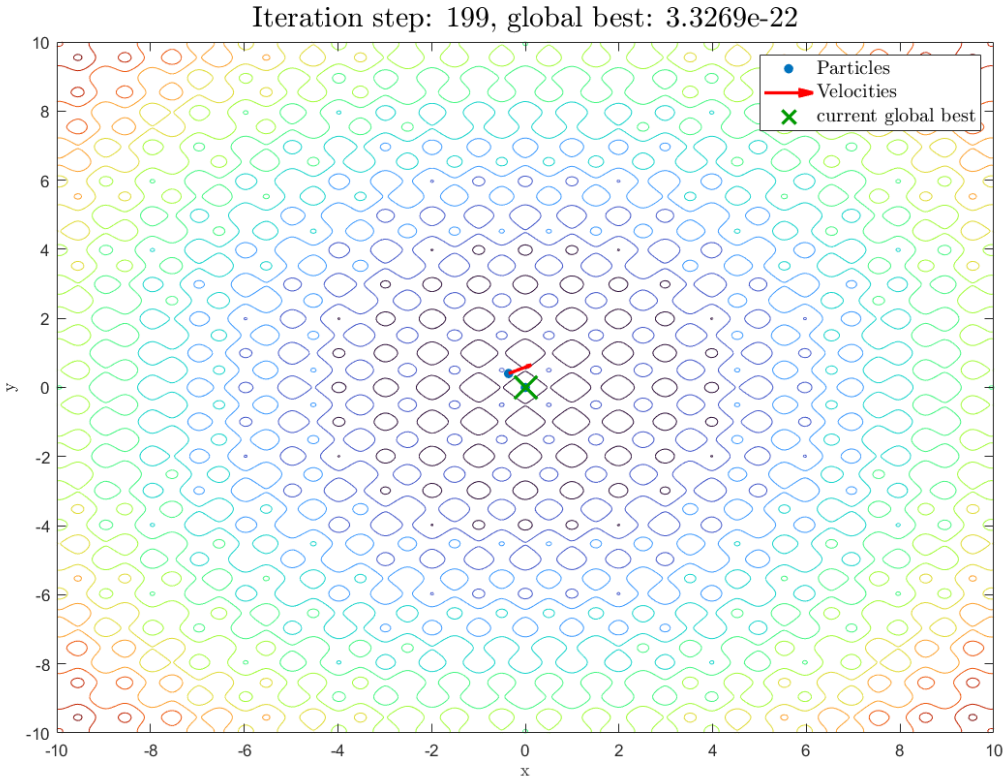

Figure 1.5: Solution 1 f1 -Graphic

Name	Value
ans	[-2.5522e-12;-1.8060e-11]
c_g	1.5000
c_p	1.5000
f1	@(x,y)x.^2+y.^2+10*(2-cos(2*pi*x)-cos(2...
g_plot	*1x1 Line*
inertia	0.7000
max_it	200
max_noupd	20
num_part	30
p0	*2x30 double*
p_plot	*1x1 Line*
pmax	10
t_plot	*1x1 Text*
v0	*2x30 double*
v_plot	*1x1 Quiver*
vmax	4
x	*2001x2001 double*
y	*2001x2001 double*
z	*2001x2001 double*

Figure 1.6: Solution 1 f1

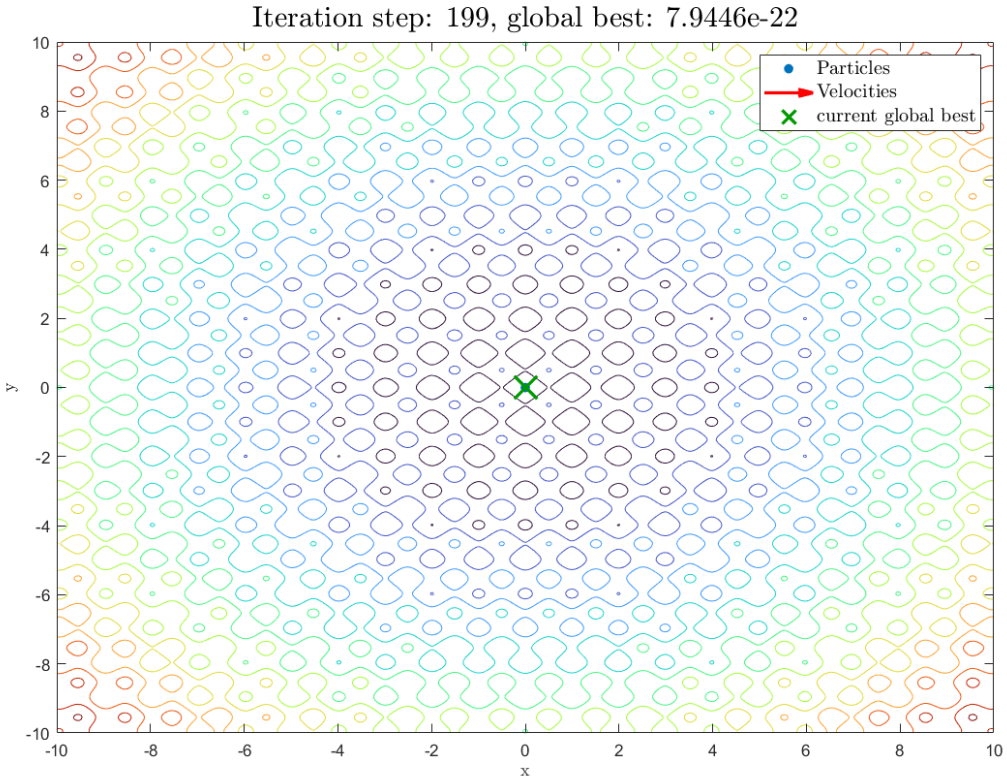

Figure 1.7: Solution 2 f1 - graphic

Name	Value
ans	[-1.2982e-11;-2.5018e-11]
c_g	1.5000
c_p	1.5000
f1	@(x,y)x.^2+y.^2+10*(2-cos(2*pi*x)-cos(2...
g_plot	*1x1 Line*
inertia	0.7000
max_it	200
max_noupd	20
num_part	30
p0	*2x30 double*
p_plot	*1x1 Line*
pmax	10
t_plot	*1x1 Text*
v0	*2x30 double*
v_plot	*1x1 Quiver*
vmax	4
x	*2001x2001 double*
y	*2001x2001 double*
z	*2001x2001 double*

Figure 1.8: Solution 2 f1 - Grahpic

1.3.4 Part d)

The PSO algorithm applied to f_2 shows that the particles often move between several local minima. This occurs because f_2 has many minima that are very close to each other in the z-dimension (function value).

As a result, these minima have almost the same depth, and the algorithm may jump from one minimum to another depending on the random initialization of the particles. Therefore, the final position changes slightly between runs, even though all solutions correspond to nearly identical objective values.

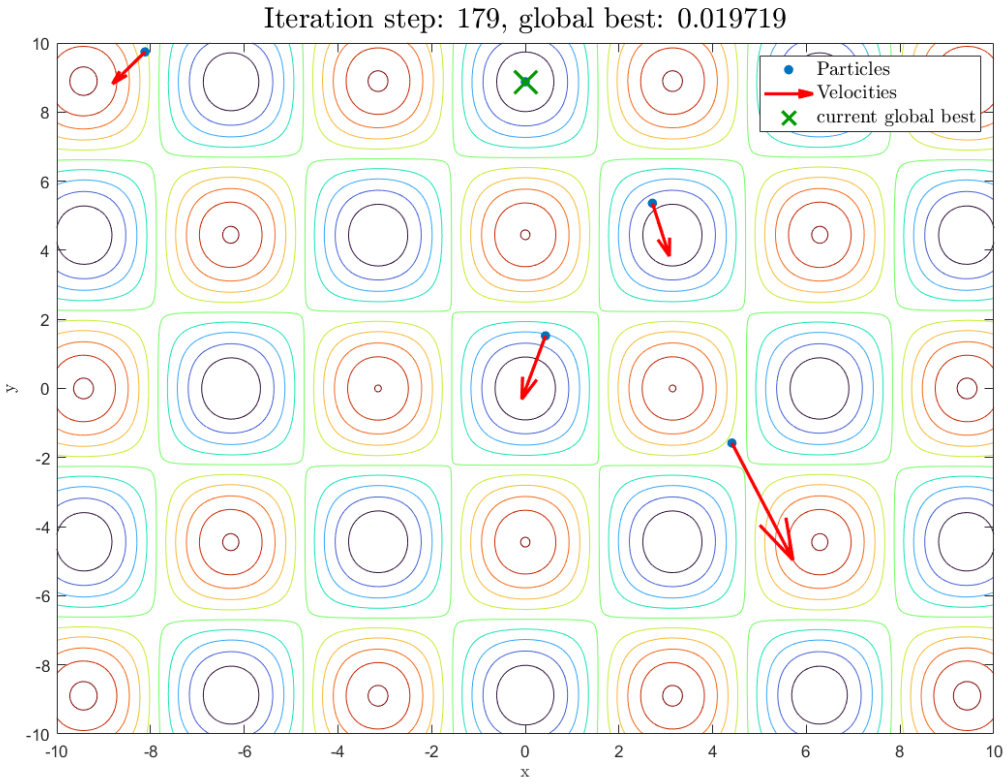

Figure 1.9: Solution 1 f2 -Graphic

Name	Value
ans	[-1.8366e-09;8.8769]
c_g	1.5000
c_p	1.5000
f2	@(x,y)1+(x.^2+y.^2)/4000-cos(x).*cos(y/...
g_plot	*1x1 Line*
inertia	0.7000
max_it	200
max_noupd	20
num_part	30
p0	*2x30 double*
p_plot	*1x1 Line*
pmax	10
t_plot	*1x1 Text*
v0	*2x30 double*
v_plot	*1x1 Quiver*
vmax	4
x	*2001x2001 double*
y	*2001x2001 double*
z	*2001x2001 double*

Figure 1.10: Solution 1 f2

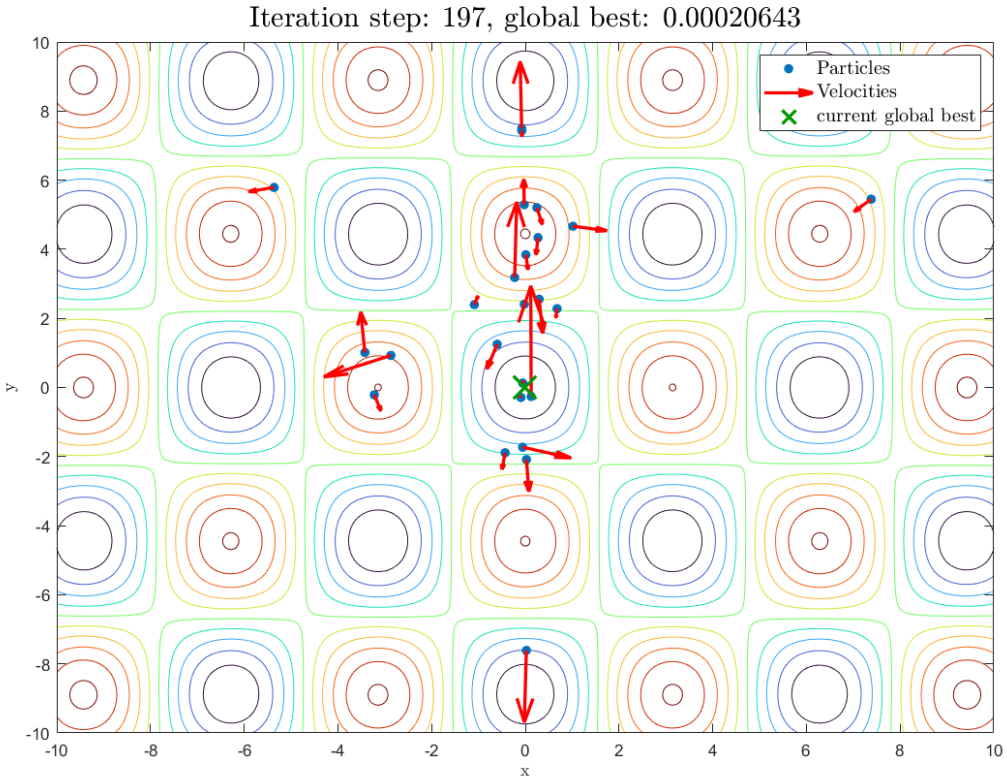

Figure 1.11: Solution 2 f2 - graphic

Name	Value
ans	[-0.0199;-0.0056]
c_g	1.5000
c_p	1.5000
f2	@(x,y)1+(x.^2+y.^2)/4000-cos(x).*cos(y/...
g_plot	*1x1 Line*
inertia	0.7000
max_it	200
max_noupd	20
num_part	30
p0	*2x30 double*
p_plot	*1x1 Line*
pmax	10
t_plot	*1x1 Text*
v0	*2x30 double*
v_plot	*1x1 Quiver*
vmax	4
x	*2001x2001 double*
y	*2001x2001 double*
z	*2001x2001 double*

Figure 1.12: Solution 2 f2 - Grahpic

1.3.5 Part e)

Although the contour plot of $f_2(x, y)$ shows many valleys of almost identical depth, the function has a single global minimum at $(0, 0)$ with $f_2(0, 0) = 0$. All other minima are slightly higher because of the quadratic term $\frac{x^2+y^2}{4000}$, but the difference is so small that they appear visually identical. As a result, the

PSO algorithm often converges to different local minima with nearly the same function value, depending on the random initial positions of the particles.

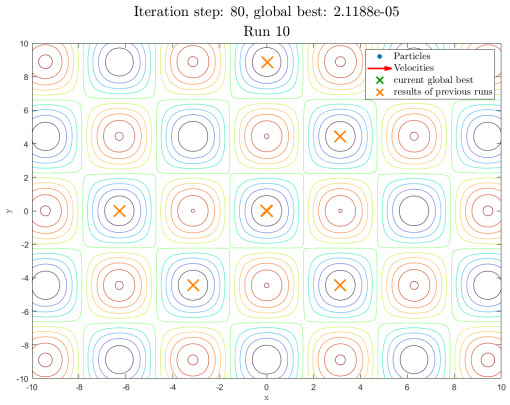

Figure 1.13: Solution 2 f2 iterations- Grahpic

1.3.6 Part f)

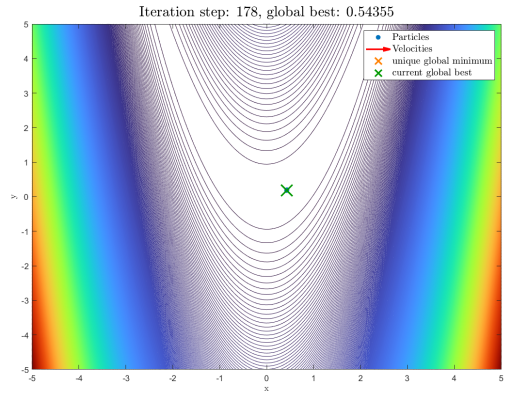

Figure 1.14: Application of particle swarm optimization for Rosenbrock

When applying the PSO algorithm to the Rosenbrock function, the global minimum was successfully found at the same point as with the Gauss–Newton method. However, PSO required about 178 iterations, significantly more than Gauss–Newton.
This difference can be explained by the nature of both methods: PSO is a population-based heuristic that explores the search space globally and does not use gradient information. Therefore, it converges more slowly but is less likely to get trapped in local minima. In contrast, the Gauss–Newton method uses local derivative information, allowing faster convergence near the optimum but relying strongly on a good initial guess.

Chapter 2

Supporting Manual Notes

Exercise sheet 2.

Exercise 1

remembering Taylor

$$f(x_k+s) \approx f(x_k) + f'(x_k)s + \frac{1}{2}f''(x_k)s^2$$

$$\frac{d}{ds} \approx 0$$

$$0 = f'(x_k) + f''(x_k)s$$

$$s_k = \frac{-f'(x_k)}{f''(x_k)} \qquad \underline{s = x - x_k}$$

$$x_{k+1} = x_k + s_k$$

multivariable

$$\underbrace{\nabla(x_k+s)}_{0} \approx \nabla f(x_k) + H(x_k).s$$

$$0 \approx \nabla f(x_k) + H(x_k)s_k \qquad x_{k+1} = x_k + s_k$$

$$s_k = -H(x_k)^{-1}.\nabla f(x_k)$$

$$x_{k+1} = x_k + s_k$$

* Gaus Newton (GN)

$$H = J^T.J$$

* Levenberg-Marquardt (LM)

$$H \approx J^T.J + \mu^2 I$$

$$s_k = -(J^T.J + \mu^2 I)^{-1} J^T F$$

$$(J^T.J + \mu^2 I)s = -J^T F$$

$$\min_{s \in \mathbb{R}^n} \|Js + F\|_2^2 + \mu^2 \|s\|_2^2$$

Exercise sheet 2 for Numerical Optimization

Dr. Michael Lehn Tobias Born

Deadline: October 29, 2025

Exercise 1 (**Levenberg-Marquardt method**, 5+5+4+4+6+2 points)

We are still dealing with nonlinear least squares problems. For their definition and the notation, see exercise sheet 1. We will now introduce a second approach for nonlinear least squares problems, the Levenberg-Marquardt method. It corresponds to the Gauß-Newton method with the slight difference, that $g''(x_k)$ is approximated in a different way. Analytically, it is

$$g''(x_k) = F'(x_k)^T F'(x_k) + \sum_{l=1}^{m} F_l(x_k) F_l''(x_k) \ .$$

The Gauß-Newton approximation is

$$g''_{(\mathrm{GN})}(x_k) = F'(x_k)^T F'(x_k) \ . \tag{1}$$

For the Levenberg-Marquardt method, we define

$$g''_{(\mathrm{LM})}(x_k) = F'(x_k)^T F'(x_k) + \mu^2 I_n$$

with $\mu \geq 0$. Therefore, the Newton correction turns out to be

$$s_k = -\left(F'(x_k)^T F'(x_k) + \mu^2 I_n\right)^{-1} F'(x_k)^T F(x_k) \ . \tag{2}$$

a) Analogously to sheet 1 exercise 3b, to which linear least squares problem is (2) equivalent?

b) Show that this linear least squares problem corresponds to the linear least squares problem (8) from sheet 1 plus a penalty term on s_k.

For $\mu = 0$ in every iteration step, the linear least squares problem (8) from sheet 1 is solved and the Gauß-Newton method executed (obviously). For $\mu > 0$, the same idea as for Gauß-Newton is adopted, but at the same time, the Newton correction is prevented from becoming too large. The basic idea is that one supposes that the error caused by approximating $g''(x_k)$ with $g''_{(\mathrm{GN})}(x_k)$, see (1), has no significant impact in a neighbourhood of x_k. Or, conversely, that the approximation error has a significant impact if ones moves too far away from x_k. The penalty term is intended to ensure that s_k is small enough so that $x_{k+1} = x_k + s_k$ is so close to x_k that the approximation error is acceptably small.

c) Show the damping property of μ on the update s_k by proving that

$$\|s_k\|_2 \leq \frac{\|F(x_k)\|_2}{\mu} \ .$$

So the bigger μ is, the bigger is the penalty term and the smaller is the neighbourhood in which x_{k+1} can be.

d) For real matrices A it holds true that $\mathrm{rank}(A) = \mathrm{rank}(A^T A)$. So for the Gauß-Newton method, it is necessary that $F'(x_k) \in \mathbb{R}^{m \times n}$ with $m \geq n$ has rank n in order to guarantee that the $(n \times n)$-dimensional system matrix $g''_{(\mathrm{GN})}(x_k)$, (1), for the update is invertible. This has to be assumed to hold true for every iterate x_k, $k = 0, 1, \ldots$.
How is it with the Levenberg-Marquardt method, i.e. for $\mu > 0$?

Now the question arises as to how the damping parameter μ should be selected. If it is too small, the iteration can leave the trusted neighbourhood and the approximation potentially becomes bad. If it is too big, the update is very small and the iteration progresses slowly. So it is about finding a good trade-off. At the same time, there is no reason to assume that a choice for μ that was a good choice in one iteration step will also be a good choice in the following iteration step. For this reason, often a damping strategy is used, the principle of which is very similar to the damping used in the damped Newton's method (recall Numerical Analysis).
To do this, for a potential update s_k, it is examined how large of an error the linearization produces. The change in the objective function induced by the update s_k

$$\|F(x_k)\|_2^2 - \|F(x_k + s_k)\|_2^2$$

is compared with the change in the linearization of the objective function

$$\|F(x_k)\|_2^2 - \|F(x_k) + F'(x_k)s_k\|_2^2 \ .$$

To this end, define the indicator

$$\epsilon_\mu := \frac{\|F(xk)\|_2^2 - \|F(x_k + s_k)\|_2^2}{\|F(x_k)\|_2^2 - \|F(x_k) + F'(x_k)s_k\|_2^2} \ . \tag{3}$$

If s_k is a reasonable choice for the update, it generates an improvement in both the actual and the linearized objective function, so we assume that $\epsilon_\mu > 0$. If the linearization was exact, it would be $\epsilon_\mu = 1$. The basic idea is as follows:

- If ϵ_μ is very small, the linearization error is too large and the update is rejected. Increase μ in order to reduce the trusted neighbourhood to enforce a better approximation error.
- Otherwise, accept the update.
- If ϵ_μ is very large, the linearization error is so small that the trusted neighbourhood can be enlarged to allow larger steps, therefore decrease μ.

With $0 < \beta_0 < \beta_1 < 1$ this means:

- $\epsilon_\mu \leq \beta_0$: reject s_k, set $\mu \leftarrow 2\mu$
- $\beta_0 < \epsilon_\mu < \beta_1$: accept s_k
- $\epsilon_\mu \geq \beta_1$: accept s_k, set $\mu \leftarrow \mu/2$

The parameters β_0 and β_1 are purely heuristic and must be selected a priori. The resulting algorithm can be seen in Algorithm 1, keep in mind that it is heuristic.

Algorithm 1: Levenberg-Marquardt method

Input: initial value $x_0 \in \mathbb{R}^n$, $F : \mathbb{R}^n \to \mathbb{R}^m$, $F' : \mathbb{R}^n \to \mathbb{R}^{m \times n}$, initial damping parameter μ_0, β_0, β_1
Output: approximation x_k of a solution to the respective nonlinear least squares problem
for $k \leftarrow 0, 1, \ldots$ **do**
  compute $F_{(k)} \leftarrow F(x_k)$ and $F'_{(k)} \leftarrow F'(x_k)$
  solve system of linear equations $s_k \leftarrow -\left(\left(F'_{(k)}\right)^T F'_{(k)} + \mu^2 I_n\right)^{-1} \left(F'_{(k)}\right)^T F_{(k)}$
  compute ϵ_μ according to (3)
  while $\epsilon_\mu \leq \beta_0$ **do**
    $\mu \leftarrow 2\mu$
    solve system of linear equations $s_k \leftarrow -\left(\left(F'_{(k)}\right)^T F'_{(k)} + \mu^2 I_n\right)^{-1} \left(F'_{(k)}\right)^T F_{(k)}$
    compute ϵ_μ according to (3)
  if $\epsilon_\mu \geq \beta_1$ **then**
    $\mu \leftarrow \mu/2$
  $x_{k+1} \leftarrow x_k + s_k$

e) Create a MATLAB programme `levenberg_marquardt.m` that gets inputs `x0`, function handles `F` and `dF`, an initial damping parameter `mu0`, `beta0`, `beta1`, a tolerance `tol` as well as a maximum number of iteration steps `maxit` and implements the Levenberg-Marquardt method. The programme is supposed to terminate when either $\|s_k\|_2 <$`tol` or when `maxit` iteration steps have been carried out.
Note: You can use the MATLAB operator \in order to solve the system of linear equations. But keep in mind that a big part of the lecture Numerical Linear Algebra dealt with this kind of problems.
Note: The programme must return a matrix of dimension ($n \times$ number iterations + 1) containing all iterates. Also it must return a (number iterations + 1)-dimensional vector containing the applied damping parameters of all iteration steps.

f) The MATLAB programmes `test_rosenbrock.m` and `error_rosenbrock.m` correspond to the programmes of the same name in sheet 1, except that this time the Rosenbrock function is optimized with both Gauß-Newton and Levenberg-Marquardt and that also the damping parameters of Levenberg-Marquardt are plotted over the iteration steps. The optimizations are performed for two different initial values. The parameters for the damping strategy of Levenberg-Marquardt are chosen as

$$\mu_0 = 0.3 \ , \quad \beta_0 = 0.3 \ , \quad \beta_1 = 0.9 \ .$$

Execute the programmes and reflect on the plots.

Exercise 2 (**Newton-Raphson method for easy examples**, 6+2 points)

By leaving the update s_k as it comes from Newton's method, i.e.

$$s_k = -\left(F'(x_k)^T F'(x_k) + \sum_{l=1}^{m} F_l(x_k) F_l''(x_k) \right)^{-1} F'(x_k)^T F(x_k) \ ,$$

one gets the Newton-Raphson method. In complicated situations, we do not want to use this method due to the m Hessian matrices. But we will take a look at it for very easy examples with one-dimensional functions $F : \mathbb{R} \to \mathbb{R}$.

a) Create a MATLAB programme `newton_raphson.m` that gets a starting point `x0`, a function handle `F`, a function handle `dF` for the derivative, a function handle `ddF` for the second derivative, a tolerance `tol` and a maximum number of iteration steps `maxit` and implements the Newton-Raphson method. The termination criterion is the same as for the Levenberg-Marquardt method. The programme is supposed to return a vector containing all iterates.

b) The MATLAB programme `test_newton_raphson.m` applies Gauß-Newton, Levenberg-Marquardt and Newton-Raphson to two simple examples,

$$f_1(x) = \|x^2 + 1\|_2^2 \quad \text{and} \quad f_2(x) = \|x^2\|_2^2 \ .$$

Run the programme and reflect on the results. In particular, focus on the behaviour of Newton-Raphson.

Exercise 3 (**Heuristic approaches: Particle Swarm Optimization**, 6+1+2+2+2+2+1)

Numerical optimization has a fundamental problem. Unless the function under consideration has special properties (e.g. convexity, ...), it is impossible to determine its global minimum with complete certainty. Take an arbitrary function and try to determine its global minimum. No matter what method you use and what the result is, how can you be sure that there is no point somewhere on the domain where the function adopts an even smaller value? For this reason, we will only deal with local optimization within the lecture, i.e., the search for local minima.
If one still wants to try to find a point where the function is globally as small as possible, you have no choice but to use heuristic approaches. These are based on ideas that sometimes work better and sometimes work worse. Except for special cases, it is not possible to figure out how wrong their results are. So, evaluating their results is less a question of mathematics and more a question of experience. Nevertheless, these approaches are common and widespread, as there is no other option. In practice, these approaches are normally used as a way to find promising starting values for local optimizations.
To enrich the lecture and so that you have seen such approaches, we will occasionally look at such methods on the exercise sheets, detached from the context of the lecture.

On this sheet, we will take a look at an algorithm called *Particle Swarm Optimization*. The name comes from the fact that it is intended to mimic the behaviour of animal swarms in a simple way. The actors are a population of particles that move around the search area. In each iteration step, each particle moves depending on its previous movement direction, the best function value it has found so far and the best function value that the entire population has found so far.
The procedure is as follows: First, random starting positions $p_0^{(1)}, \dots, p_0^{(n)}$ and random starting velocities $v_0^{(1)}, \dots, v_0^{(n)}$ are drawn for n particles. For each particle, a record is kept of its position with the best function value across all iterations. At each iteration step, it must therefore be checked whether the new position has a function value that is better than the particle's former best position. The minimum of the i-th particle up to the k-th iteration step is denoted by $\overline{p}_k^{(i)}$. Similarly, the current minimum across all particles after k iteration steps, i.e., the current global minimum, is denoted by $\overline{g}_k$ and must be updated in each iteration step. In the k-th iteration step, the velocity $v_k^{(i)}$ is determined for the i-th particle. It is composed of the previous velocity multiplied by an inertia ω, the distance of the particle to its minimum so far multiplied with a specified weight c_p and a random weight and the distance of the particle to the global minimum so far multiplied with a specified weight c_g and a random weight. The new position $p_k^{(i)}$ is then determined by adding the velocity $v_k^{(i)}$ to the old position $p_{k-1}^{(i)}$.

Algorithm 2: Particle Swarm Optimization

Input: initial positions $p_0^{(i)}$ and velocities $v_0^{(i)}$, $i = 1, \dots, n$, inertia ω, weights c_p and c_g
Output: approximation of the global minimum $\overline{g}_k$
for $k \leftarrow 1, 2, \dots$ **do**
  update $\overline{p}_{k-1}^{(i)}$, $\forall i = 1, \dots, n$ // particle's best position over all iteration steps
  determine $\overline{g}_{k-1}$ // global best position over all iteration steps
  $v_k^{(i)} \leftarrow \omega v_{k-1}^{(i)} + \text{rand} \cdot c_p \left(\overline{p}_{k-1}^{(i)} - p_{k-1}^{(i)}\right) + \text{rand} \cdot c_g \left(\overline{g}_{k-1} - p_{k-1}^{(i)}\right)$, $\forall i = 1, \dots, n$
    where rand is a uniformly distributed random number in $(0, 1)$
  $p_k^{(i)} = p_{k-1}^{(i)} + v_k^{(i)}$, $\forall i = 1, \dots, n$

The algorithm should terminate when either a maximum number of iteration steps has been reached (`max_it`) or the global optimum $\overline{g}_k$ has not changed for a certain number of iteration steps (`max_noupd`).

a) We want to implement the Particle Swarm Optimization. Therefore, fill the gaps in the MATLAB programme `particle_swarm_optimization.m`.

We want to try out `particle_swarm_optimization.m` on the functions

$$f_1(x,y) = x^2 + y^2 + 10\,(2 - \cos(2\pi x) - \cos(2\pi y)) \quad \text{and}$$
$$f_2(x,y) = 1 + \frac{x^2+y^2}{4000} - \cos(x)\cos\left(\frac{y}{\sqrt{2}}\right) \ .$$

b) Execute the MATLAB programme `visualize_functions.m` and understand what f_1 and f_2 look like and which of their properties could be relevant for optimization.

From now on, the heuristic parameters are always selected as $\omega = 0.7$, $c_p = 1.5$, $c_g = 1.5$, `max_it` $= 200$ and `max_noupd` $= 20$. Keep in mind that these parameters could also be selected completely differently and that the algorithm may then behave completely differently.

c) The MATLAB programme `test_pso_f1.m` draws 30 random particles, uniformly distributed on $(-10,10)\times(-10,10)$, in combination with random initial velocities uniformly distributed on $(-4,4)\times(-4,4)$. Then it tries to optimize f_1 using `particle_swarm_optimization.m` and plots the results.
Run the programme several times and reflect on the outcome.

d) The MATLAB programme `test_pso_f2.m` does exactly the same with f_2. Run it several times and reflect on the outcome.

As we have no information about the error of the method, it is not useful to perform it only once. Instead, the method will be performed several times for different random initial values and the results will be compared.

e) The MATLAB programme `test_pso_f2_repeat.m` does the same as `test_pso_f2.m`, with the difference that it performs ten independent optimization runs, each with different random starting values.
Run the programme and reflect on the outcome.

f) Finally, let us also take a look at the Rosenbrock function. The MATLAB programme `test_pso_rosenbrock.m` applies `particle_swarm_optimization.m` with 30 uniformly distributed initial positions in $(-5,5)\times(-5,5)$ and uniformly distributed initial velocities in $(-2,2)\times(-2,2)$ to the Rosenbrock function.
Run the programme and reflect on the outcome.

g) Finally, I encourage you to play around with the heuristic parameters of the Particle Swarm Optimization to get a feeling of how different it behaves for differing choices of parameters.